

AiPerm智能权限管理软件 源代码文档

基本信息

项目	内容
软件名称	AiPerm智能权限管理软件
版本号	V1.0
著作权人	河南爱编程网络科技有限公司
总代码行数	约38,000行
提取代码行数	3,000行（前后各1,500行）
页数	60页（前后各30页）
每页行数	50行

前30页（第1页-第30页）

```
=====
文件: frontend/src/main.ts
=====
```

```
import {createApp} from 'vue'
import {createPinia} from 'pinia'
import piniaPluginPersistedstate from 'pinia-plugin-persistedstate'
import ElementPlus from 'element-plus'
import zhCn from 'element-plus/es/locale/lang/zh-cn'
import 'element-plus/dist/index.css'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'
import 'uno.css'
import './style.css'

import App from './App.vue'
import {setupRouter} from './router'
import permissionDirectives from './directives/permission'
import DictTag from './components/dict/DictTag.vue'
import DictSelect from './components/dict/DictSelect.vue'
import DictRadio from './components/dict/DictRadio.vue'
import TableToolBar from './components/table/TableToolBar.vue'
import ColumnSetting from './components/table/ColumnSetting.vue'
import SelectionBar from './components/table/SelectionBar.vue'

const app = createApp(App)

// 注册 Element Plus 图标
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}

// 创建 Pinia 实例并配置持久化插件
const pinia = createPinia()
pinia.use(piniaPluginPersistedstate)

app.use(pinia)
app.use(ElementPlus, { locale: zhCn })

// 注册全局组件
app.component('DictTag', DictTag)
app.component('DictSelect', DictSelect)
app.component('DictRadio', DictRadio)
app.component('TableToolBar', TableToolBar)
app.component('ColumnSetting', ColumnSetting)
app.component('SelectionBar', SelectionBar)

// 注册权限指令
app.directive('permission', permissionDirectives.permission)
app.directive('role', permissionDirectives.role)

setupRouter(app)
```

```
app.mount('#app')
```

```
=====
文件: frontend/src/App.vue
=====
```

```
<script setup lang="ts">
import {onMounted, onUnmounted, ref} from 'vue'
import {RouterView} from 'vue-router'
import {useAppStore} from '@stores/app'
import MfaVerifyDialog from '@components/mfa/MfaVerifyDialog.vue'

const appStore = useAppStore()
const mfaDialogVisible = ref(false)

// 处理 2FA 验证请求
function handleMfaRequired() {
  mfaDialogVisible.value = true
}

// 应用初始化时恢复主题
onMounted(() => {
  // 恢复保存的主题状态
  appStore.updateThemeClass()
  // 监听 2FA 验证请求事件
  window.addEventListener('mfa-required', handleMfaRequired)
})

onUnmounted(() => {
  window.removeEventListener('mfa-required', handleMfaRequired)
})
</script>

<template>
  <RouterView />
  <!-- 全局 2FA 验证弹窗 -->
  <MfaVerifyDialog
    v-model:visible="mfaDialogVisible"
    @success="mfaDialogVisible = false"
  />
</template>

<style>
#app {
  width: 100%;
  height: 100vh;
}
</style>
```

```
=====
文件: frontend/src/types/index.ts
=====
```

```
/**
 * 通用类型定义
 * 与后端 common.domain 包中的类型对应
 */

// 统一响应结构 R<T>
export interface R<T = any> {
  code: number
  message: string
  data: T
}

// 分页结果
export interface PageResult<T> {
  total: number
  list: T[]
  pageNum: number
  pageSize: number
  pages: number
}

// 分页请求参数
export interface PageParams {
  page?: number
  pageSize?: number
}

export interface ImportError {
  rowNum: number
  message: string
}

export interface ImportResult {
  successCount: number
  failureCount: number
  errors: ImportError[]
}

// 表格列配置（用于 ColumnSetting 组件）
export interface TableColumn {
  key: string // 对应 el-table-column 的 prop
  label: string // 列标题
  visible: boolean // 是否显示
  fixed?: boolean | 'left' | 'right' // 固定列: true/'left'为左固定, 'right'为右固定（固定列不允许隐藏）
}
```

=====

文件: frontend/src/router/index.ts

=====

```
import {createRouter, createWebHistory, type Router, type RouteRecordRaw} from 'vue-router'
import type {App} from 'vue'
import {useUserStore} from '@stores/user'
import {usePermissionStore} from '@stores/permission'
```

// 静态路由（不需要权限）

```
export const constantRoutes: RouteRecordRaw[] = [
  {
    path: '/login',
    name: 'Login',
    component: () => import('@views/login/index.vue'),
    meta: {
      title: '登录',
      requiresAuth: false,
    },
  },
  {
    path: '/404',
    name: 'NotFound',
    component: () => import('@views/error/404.vue'),
    meta: {
      title: '404',
    },
  },
  {
    path: '/',
    name: 'Root',
    component: () => import('@components/layout/MainLayout.vue'),
    redirect: '/dashboard',
    children: [
      {
        path: 'dashboard',
        name: 'Dashboard',
        component: () => import('@views/dashboard/index.vue'),
        meta: {
          title: '首页',
          icon: 'HomeFilled',
          requiresAuth: true,
        },
      },
      {
        path: 'profile',
        name: 'Profile',
        component: () => import('@views/profile/index.vue'),
        meta: {
          title: '个人中心',
        },
      },
    ],
  },
]
```

```

        icon: 'User',
        requiresAuth: true,
      },
    ],
  },
]

// 创建路由实例
const router: Router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: constantRoutes,
  scrollBehavior: () => ({ top: 0 }),
})

// 白名单路由
const whiteList = ['/login', '/404']

// 设置路由守卫
function setupRouterGuard(router: Router) {
  router.beforeEach(async (to, _from, next) => {
    const userStore = useUserStore()
    const permissionStore = usePermissionStore()
    const token = userStore.token

    // 设置页面标题
    document.title = `${to.meta.title ?? 'AIPerm'} - 权限管理系统`

    // 未登录
    if (!token) {
      if (whiteList.includes(to.path)) {
        next()
      } else {
        next(`/login?redirect=${to.path}`)
      }
      return
    }

    // 已登录访问登录页
    if (to.path === '/login') {
      next('/')
      return
    }

    // 路由已加载
    if (permissionStore.isRoutesLoaded) {
      next()
      return
    }
  })
}

```

```

// 动态加载路由
try {
  // 1. 获取用户信息
  await userStore.fetchUserInfo()

  // 2. 获取菜单
  await permissionStore.fetchMenus()

  // 3. 生成路由
  const routes = permissionStore.generateRoutes()

  // 4. 动态添加路由（添加为 Root 路由的子路由）
  routes.forEach((route) => {
    router.addRoute('Root', route)
  })

  // 5. 添加 404 兜底路由（必须最后添加）
  router.addRoute({
    path: '/:pathMatch(.*)*',
    redirect: '/404',
  })

  // 6. 重新导航
  next({ ...to, replace: true })
}
catch (error) {
  console.error('Failed to load routes:', error)
  // 加载失败，清除 token 并跳转登录
  userStore.logout()
  next(`/login?redirect=${to.path}`)
}
})

router.afterEach(() => {
  // 路由切换后处理
})
}

// 初始化路由
export function setupRouter(app: App) {
  setupRouterGuard(router)
  app.use(router)
}

export default router

```

```

=====
文件: frontend/src/utils/request.ts

```

```

=====
/**
 * 请求工具
 * 基于现有的 axiosInstance 封装
 */
import type {AxiosRequestConfig} from 'axios'
import {axiosInstance} from './api-mutator'
import type {R} from '@types'

function resolveFilename(contentDisposition?: string, fallback = 'download.xlsx') {
  if (!contentDisposition) {
    return fallback
  }

  const utf8Match = contentDisposition.match(/filename*=UTF-8'([^\s;]+)'/i)
  if (utf8Match?.[1]) {
    return decodeURIComponent(utf8Match[1])
  }

  const match = contentDisposition.match(/filename="([^\s"]+)"?/i)
  if (match?.[1]) {
    return decodeURIComponent(match[1])
  }

  return fallback
}

function triggerDownload(blob: Blob, filename: string) {
  const url = window.URL.createObjectURL(blob)
  const link = document.createElement('a')
  link.href = url
  link.download = filename
  document.body.appendChild(link)
  link.click()
  document.body.removeChild(link)
  window.URL.revokeObjectURL(url)
}

// 请求封装
const request = {
  get: <T>(url: string, config?: any): Promise<T> =>
    axiosInstance.get<R<T>>(url, config).then(res => res.data.data),

  post: <T>(url: string, data?: any, config?: any): Promise<T> =>
    axiosInstance.post<R<T>>(url, data, config).then(res => res.data.data),

  put: <T>(url: string, data?: any, config?: any): Promise<T> =>
    axiosInstance.put<R<T>>(url, data, config).then(res => res.data.data),

  delete: <T>(url: string, config?: any): Promise<T> =>

```

```

    axiosInstance.delete<R<T>>>(url, config).then(res => res.data.data),

    upload: <T>(url: string, file: File, fieldName = 'file'): Promise<T> => {
      const formData = new FormData()
      formData.append(fieldName, file)
      return axiosInstance.post<R<T>>>(url, formData, {
        headers: { 'Content-Type': 'multipart/form-data' },
      }).then(res => res.data.data)
    },

    download: async (url: string, config?: AxiosRequestConfig, fallbackFilename?: string): Promise<void> => {
      const response = await axiosInstance.get(url, {
        ...config,
        responseType: 'blob',
      })
      const filename = resolveFilename(response.headers['content-disposition'], fallbackFilename)
      const blob = response.data instanceof Blob
        ? response.data
        : new Blob([response.data], { type: response.headers['content-type'] })
      triggerDownload(blob, filename || 'download.xlsx')
    },
  },
}

export default request

=====
文件: frontend/src/stores/user.ts
=====

import {defineStore} from 'pinia'
import {computed, ref, shallowRef} from 'vue'
import {authApi} from '@/api/auth'

export interface UserInfo {
  id: number
  username: string
  nickname?: string
  email?: string
  phone?: string
  avatar?: string
  roles: string[]
  permissions: string[]
}

export const useUserStore = defineStore(
  'user',
  () => {
    // 使用 shallowRef 优化大对象性能
    const token = ref<string | null>(null)
    const userInfo = shallowRef<UserInfo | null>(null)
  }
)

```

```

// 计算属性
const isLoggedIn = computed(() => !!token.value)
const username = computed(() => userInfo.value?.username ?? "")
const roles = computed(() => userInfo.value?.roles ?? [])
const permissions = computed(() => userInfo.value?.permissions ?? [])

// 设置 Token
function setToken(newToken: string) {
  token.value = newToken
}

// 设置用户信息
function setUserInfo(info: UserInfo) {
  userInfo.value = info
}

// 从后端获取用户信息
async function fetchUserInfo() {
  const data = await authApi.info()
  userInfo.value = {
    id: data.id,
    username: data.username,
    nickname: data.nickname,
    avatar: data.avatar,
    roles: data.roles,
    permissions: data.permissions,
  }
  return userInfo.value
}

// 检查是否有指定角色
function hasRole(role: string): boolean {
  return roles.value.includes(role)
}

// 检查是否有指定权限
function hasPermission(permission: string): boolean {
  return permissions.value.includes(permission)
}

// 检查是否有任一权限
function hasAnyPermission(permissionList: string[]): boolean {
  return permissionList.some(p => hasPermission(p))
}

// 登出
async function logout() {
  try {
    await authApi.logout()
  }
}

```

```

    }
    catch {
      // 忽略登出错误
    }
    finally {
      token.value = null
      userInfo.value = null
    }
  }
}

return {
  // 状态
  token,
  userInfo,
  // 计算属性
  isLoggedIn,
  username,
  roles,
  permissions,
  // 方法
  setToken,
  setUserInfo,
  fetchUserInfo,
  hasRole,
  hasPermission,
  hasAnyPermission,
  logout,
}
},
{
  persist: {
    key: 'aiperm-user',
    storage: localStorage,
    pick: ['token', 'userInfo'],
  },
},
)

```

```

=====
文件: frontend/src/api/system/user.ts
=====

```

```

/**
 * 用户管理 API
 * 对应后端 SysUserController (/system/user)
 */
import request from '@/utils/request'
import type {ImportResult, PageParams, PageResult} from '@/types'

/** 用户实体 */

```

```
export interface UserVO {
  id: number
  username: string
  nickname: string
  realName?: string
  email: string
  phone: string
  gender: number
  avatar: string
  status: number
  deptId: number
  deptName: string
  postId?: number
  postIds: number[]
  postNames: string
  roleIds: number[]
  roleNames: string
  createTime: string
  createBy: string
  updateTime: string
  updateBy: string
}

/** 用户查询/创建/更新 DTO */
export interface UserDTO extends PageParams {
  id?: number
  username?: string
  nickname?: string
  realName?: string
  email?: string
  phone?: string
  gender?: number
  avatar?: string
  status?: number
  deptId?: number
  postId?: number
  postIds?: number[]
  roleIds?: number[]
  password?: string
  newPassword?: string
  remark?: string
}

export const userApi = {
  list: (params: UserDTO) =>
    request.get<PageResult<UserVO>>('/system/user', { params }),

  export: (params: UserDTO) =>
    request.download('/system/user/export', { params }, 'users.xlsx'),
}
```

```

downloadImportTemplate: () =>
  request.download('/system/user/import-template', undefined, 'user-import-template.xlsx'),

importUsers: (file: File) =>
  request.upload<ImportResult>('/system/user/import', file),

getById: (id: number) =>
  request.get<UserVO>(`/system/user/${id}`),

create: (data: UserDTO) =>
  request.post<void>('/system/user', data),

update: (id: number, data: UserDTO) =>
  request.put<void>(`/system/user/${id}`, data),

delete: (id: number) =>
  request.delete<void>(`/system/user/${id}`),

deleteBatch: (ids: number[]) =>
  request.delete<void>('/system/user/batch', { data: ids }),

resetPassword: (id: number, newPassword: string) =>
  request.put<void>(`/system/user/${id}/reset-password`, { newPassword }),

changeStatus: (id: number, status: number) =>
  request.put<void>(`/system/user/${id}/status`, null, { params: { status } }),
}

```

```

=====
文件: frontend/src/api/system/role.ts
=====

```

```

/**
 * 角色管理 API
 * 对应后端 SysRoleController (/system/role)
 */
import request from '@utils/request'
import type {PageParams, PageResult} from '@types'

// ===== 类型定义 =====

/** 角色实体 */
export interface RoleVO {
  id: number
  roleName: string
  roleCode: string
  sort: number
  status: number
  dataScope: number
  remark: string
}

```

```

    createTime: string
    createBy: string
    updateTime: string
    updateBy: string
  }

  /** 角色查询/创建/更新 DTO */
  export interface RoleDTO extends PageParams {
    id?: number
    roleName?: string
    roleCode?: string
    sort?: number
    status?: number
    dataScope?: number
    remark?: string
  }

  // ===== API 函数 =====

  export const roleApi = {
    /** 分页查询角色列表 */
    list: (params: RoleDTO) =>
      request.get<PageResult<RoleVO>>('/system/role', { params }),

    /** 查询所有角色 */
    all: () =>
      request.get<RoleVO[]>('/system/role/all'),

    /** 根据 ID 查询角色 */
    getById: (id: number) =>
      request.get<RoleVO>(`/system/role/${id}`),

    /** 获取角色的菜单 ID 列表 */
    getRoleMenus: (id: number) =>
      request.get<number[]>(`/system/role/${id}/menus`),

    /** 创建角色 */
    create: (data: RoleDTO) =>
      request.post<void>('/system/role', data),

    /** 更新角色 */
    update: (id: number, data: RoleDTO) =>
      request.put<void>(`/system/role/${id}`, data),

    /** 删除角色 */
    delete: (id: number) =>
      request.delete<void>(`/system/role/${id}`),

    /** 批量删除角色 */
    deleteBatch: (ids: number[]) =>

```

```

    request.delete<void>('/system/role/batch', { data: ids }),

    /** 分配角色菜单 */
    assignMenus: (id: number, menuIds: number[]) =>
        request.post<void>('/system/role/${id}/menus', menuIds),
    }

=====
文件: frontend/src/api/system/menu.ts
=====

/**
 * 菜单管理 API
 * 对应后端 SysMenuController (/system/menu)
 */
import request from '@utils/request'
import type {PageParams} from '@types'

// ===== 类型定义 =====

/** 菜单实体 */
export interface MenuVO {
    id: number
    parentId: number
    menuName: string
    menuCode: string
    path: string
    icon: string
    menuType: number | string // 1-目录 2-菜单 3-按钮 (可能是数字或字符串)
    sort: number
    status: number
    visible: number
    component: string
    perms: string
    remark: string
    isExternal: number
    isCache: number
    children?: MenuVO[]
    createTime: string
    createBy: string
    updateTime: string
    updateBy: string
}

/** 菜单查询/创建/更新 DTO */
export interface MenuDTO extends PageParams {
    id?: number
    parentId?: number
    menuName?: string
    menuCode?: string

```

```

path?: string
icon?: string
menuType?: number
sort?: number
status?: number
visible?: number
component?: string
perms?: string
remark?: string
isExternal?: number
isCache?: number
}

// ===== API 函数 =====

export const menuApi = {
  /** 查询所有菜单 */
  list: () =>
    request.get<MenuVO[]>('/system/menu'),

  /** 查询菜单树 */
  tree: () =>
    request.get<MenuVO[]>('/system/menu/tree'),

  /** 根据 ID 查询菜单 */
  getById: (id: number) =>
    request.get<MenuVO>(`/system/menu/${id}`),

  /** 查询子菜单列表 */
  getChildren: (parentId: number) =>
    request.get<MenuVO[]>(`/system/menu/children/${parentId}`),

  /** 创建菜单 */
  create: (data: MenuDTO) =>
    request.post<void>('/system/menu', data),

  /** 更新菜单 */
  update: (id: number, data: MenuDTO) =>
    request.put<void>(`/system/menu/${id}`, data),

  /** 删除菜单 */
  delete: (id: number) =>
    request.delete<void>(`/system/menu/${id}`),
}

=====
文件: frontend/src/views/login/index.vue
=====
<script setup lang="ts">

```

```

import {computed, onMounted, reactive, ref} from 'vue'
import {useRouter} from 'vue-router'
import {ElMessage, type FormInstance, type FormRules} from 'element-plus'
import {Iphone, Lock, Message, Picture, User} from '@element-plus/icons-vue'
import {useUserStore} from '@stores/user'
import {authApi, type CaptchaVO, type LoginConfigVO, type UnifiedLoginDTO} from '@api/auth'
import {captchaApi} from '@api/captcha'

const router = useRouter()
const userStore = useUserStore()

// 表单引用
const formRef = ref<FormInstance>()

// 当前登录方式
const activeTab = ref<'PASSWORD' | 'SMS' | 'EMAIL'>('PASSWORD')

// 登录配置
const loginConfig = ref<LoginConfigVO>({
  passwordEnabled: true,
  smsEnabled: false,
  emailEnabled: false,
  oauthConfigs: [],
})

// 表单数据
const loginForm = reactive<UnifiedLoginDTO>({
  loginType: 'PASSWORD',
  identifier: '',
  credential: '',
  imageCaptcha: '',
  imageCaptchaKey: '',
})

// 验证码数据
const captchaData = ref<CaptchaVO>({
  captchaKey: '',
  captchaImage: '',
})

// 加载状态
const loading = ref(false)

// 表单验证规则 - 根据登录类型动态变化
const rules = computed<FormRules>(() => {
  const baseRules: FormRules = {
    credential: [
      { required: true, message: '请输入密码', trigger: 'blur' },
    ],
  }
})

```

```

if (activeTab.value === 'PASSWORD') {
  return {
    ...baseRules,
    identifier: [
      { required: true, message: '请输入用户名', trigger: 'blur' },
      { min: 2, max: 20, message: '用户名长度为 2-20 个字符', trigger: 'blur' },
    ],
    imageCaptcha: [
      { required: true, message: '请输入验证码', trigger: 'blur' },
      { min: 4, max: 4, message: '验证码为 4 位字符', trigger: 'blur' },
    ],
  }
}
else if (activeTab.value === 'SMS') {
  return {
    ...baseRules,
    identifier: [
      { required: true, message: '请输入手机号', trigger: 'blur' },
      { pattern: /^1[3-9]\d{9}$/, message: '请输入正确的手机号', trigger: 'blur' },
    ],
    credential: [
      { required: true, message: '请输入验证码', trigger: 'blur' },
      { len: 6, message: '验证码为 6 位数字', trigger: 'blur' },
    ],
  }
}
else if (activeTab.value === 'EMAIL') {
  return {
    ...baseRules,
    identifier: [
      { required: true, message: '请输入邮箱', trigger: 'blur' },
      { type: 'email', message: '请输入正确的邮箱地址', trigger: 'blur' },
    ],
    credential: [
      { required: true, message: '请输入验证码', trigger: 'blur' },
      { len: 6, message: '验证码为 6 位数字', trigger: 'blur' },
    ],
  }
}

return baseRules
})

// 获取登录配置
async function fetchLoginConfig() {
  try {
    const config = await authApi.loginConfig()
    if (config) {
      loginConfig.value = config
    }
  }
}

```

```

    }
  }
  catch (error) {
    console.error('获取登录配置失败:', error)
  }
}

// 获取图形验证码
async function fetchCaptcha() {
  try {
    const captchaInfo = await authApi.captcha()
    if (captchaInfo) {
      captchaData.value = {
        captchaKey: captchaInfo.captchaKey || '',
        captchaImage: captchaInfo.captchaImage || '',
      }
      loginForm.imageCaptchaKey = captchaInfo.captchaKey || ''
    }
  }
  catch (error) {
    console.error('获取验证码失败:', error)
    ElMessage.error('获取验证码失败, 请刷新页面')
  }
}

// 发送短信验证码
const smsCountdown = ref(0)
async function sendSmsCode() {
  if (!loginForm.identifier || /^[3-9]\d{9}$/.test(loginForm.identifier)) {
    ElMessage.warning('请先输入正确的手机号')
    return
  }
  if (smsCountdown.value > 0)
    return

  try {
    await captchaApi.send({
      target: loginForm.identifier,
      type: 'SMS',
      scene: 'LOGIN',
    })
    ElMessage.success('验证码已发送')
    smsCountdown.value = 60
    const timer = setInterval(() => {
      smsCountdown.value--
      if (smsCountdown.value <= 0) {
        clearInterval(timer)
      }
    }, 1000)
  }
}

```

```

    catch (error: any) {
      EMessage.error(error?.message || '发送失败')
    }
  }

  // 发送邮件验证码
  const emailCountdown = ref(0)
  async function sendEmailCode() {
    if (!loginForm.identifier || /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(loginForm.identifier)) {
      EMessage.warning('请先输入正确的邮箱地址')
      return
    }
    if (emailCountdown.value > 0)
      return

    try {
      await captchaApi.send({
        target: loginForm.identifier,
        type: 'EMAIL',
        scene: 'LOGIN',
      })
      EMessage.success('验证码已发送')
      emailCountdown.value = 60
      const timer = setInterval(() => {
        emailCountdown.value--
        if (emailCountdown.value <= 0) {
          clearInterval(timer)
        }
      }, 1000)
    }
    catch (error: any) {
      EMessage.error(error?.message || '发送失败')
    }
  }

  // 切换登录方式
  function handleTabChange(tab: 'PASSWORD' | 'SMS' | 'EMAIL') {
    activeTab.value = tab
    loginForm.loginType = tab
    loginForm.identifier = ''
    loginForm.credential = ''
    loginForm.imageCaptcha = ''
    formRef.value?.clearValidate()
  }

  // 处理登录
  async function handleLogin() {
    if (!formRef.value)
      return
  }

```

```
try {
  await formRef.value.validate()
  loading.value = true

  const loginData = await authApi.unifiedLogin(loginForm)

  if (loginData) {
    userStore.setToken(loginData.token || "")

    if (loginData.userInfo) {
      userStore.setUserInfo({
        id: loginData.userInfo.id || 0,
        username: loginData.userInfo.username || "",
        nickname: loginData.userInfo.nickname || "",
        roles: ['admin'],
        permissions: ['*'],
      })
    }
  }

  ElMessage.success('登录成功')
  router.push('/')
}
}
catch (error: any) {
  console.error('登录失败:', error)
  if (activeTab.value === 'PASSWORD') {
    fetchCaptcha()
    loginForm.imageCaptcha = ""
  }
}
finally {
  loading.value = false
}
}

// 重置表单
function resetForm() {
  formRef.value?.resetFields()
  if (activeTab.value === 'PASSWORD') {
    fetchCaptcha()
  }
}

// 页面加载时
onMounted(() => {
  fetchLoginConfig()
  fetchCaptcha()
})
</script>
```

```
<template>
<div class="login-container">
  <!-- 左侧品牌区域 -->
  <div class="login-brand">
    <div class="brand-content">
      <div class="brand-logo">
        
      </div>
      <h1 class="brand-title">
        AIPerm 权限管理系统
      </h1>
      <p class="brand-subtitle">
        专业的企业级 RBAC 权限管理解决方案
      </p>
      <div class="brand-features">
        <div class="feature-item">
          <svg
            class="feature-icon"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            stroke-width="2"
          >
            <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z" />
          </svg>
          <span>安全可靠</span>
        </div>
        <div class="feature-item">
          <svg
            class="feature-icon"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            stroke-width="2"
          >
            <path d="M13 2L3 14h9l-1 8 10-12h-9l1-8z" />
          </svg>
          <span>高效便捷</span>
        </div>
        <div class="feature-item">
          <svg
            class="feature-icon"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            stroke-width="2"
          >
```

```

    >
    <circle
      cx="12"
      cy="12"
      r="10"
    />
    <path d="M12 6v6l4 2" />
  </svg>
  <span>实时响应</span>
</div>
</div>
<div class="brand-decoration">
  <div class="decoration-circle decoration-circle-1" />
  <div class="decoration-circle decoration-circle-2" />
  <div class="decoration-circle decoration-circle-3" />
</div>
</div>

<!-- 右侧登录区域 -->
<div class="login-form-wrapper">
  <div class="login-form-container">
    <!-- 移动端 Logo -->
    <div class="mobile-logo">
      
    </div>

    <div class="login-header">
      <h2 class="login-title">
        欢迎登录
      </h2>
      <p class="login-subtitle">
        请输入您的账号信息
      </p>
    </div>

    <!-- 登录方式 Tab -->
    <div class="login-tabs">
      <button
        v-if="loginConfig.passwordEnabled"
        :class="['tab-btn', { active: activeTab === 'PASSWORD' }]"
        @click="handleTabChange('PASSWORD')"
      >
        <el-icon><User /></el-icon>
        密码登录
      </button>
    </div>
  </div>
</div>

```

```

<button
  v-if="loginConfig.smsEnabled"
  :class="['tab-btn', { active: activeTab === 'SMS' }]"
  @click="handleTabChange('SMS')"
>
  <el-icon><lphone /></el-icon>
  手机登录
</button>
<button
  v-if="loginConfig.emailEnabled"
  :class="['tab-btn', { active: activeTab === 'EMAIL' }]"
  @click="handleTabChange('EMAIL')"
>
  <el-icon><Message /></el-icon>
  邮箱登录
</button>
</div>

<el-form
  ref="formRef"
  :model="loginForm"
  :rules="rules"
  label-width="0"
  size="large"
  class="login-form"
  @keyup.enter="handleLogin"
>
  <!-- 密码登录 -->
  <template v-if="activeTab === 'PASSWORD'">
    <el-form-item prop="identifier">
      <el-input
        v-model="loginForm.identifier"
        placeholder="请输入用户名"
        :prefix-icon="User"
        clearable
      />
    </el-form-item>

    <el-form-item prop="credential">
      <el-input
        v-model="loginForm.credential"
        type="password"
        placeholder="请输入密码"
        :prefix-icon="Lock"
        show-password
      />
    </el-form-item>

    <el-form-item prop="imageCaptcha">
      <div class="captcha-row">

```

```

<el-input
  v-model="loginForm.imageCaptcha"
  placeholder="请输入验证码"
  :prefix-icon="Picture"
  clearable
  class="captcha-input"
/>
<div
  class="captcha-box"
  title="点击刷新验证码"
  @click="fetchCaptcha"
>
  
  <div
    v-else
    class="captcha-loading"
  >
    点击刷新
  </div>
</div>
</el-form-item>
</template>

<!-- 手机号登录 -->
<template v-else-if="activeTab === 'SMS'">
  <el-form-item prop="identifier">
    <el-input
      v-model="loginForm.identifier"
      placeholder="请输入手机号"
      :prefix-icon="Iphone"
      clearable
    />
  </el-form-item>

  <el-form-item prop="credential">
    <div class="captcha-row">
      <el-input
        v-model="loginForm.credential"
        placeholder="请输入验证码"
        :prefix-icon="Picture"
        clearable
        class="captcha-input"
        maxlength="6"
      />
    </div>
  </el-form-item>

```

```

      <el-button
        :disabled="smsCountdown > 0"
        class="send-btn"
        @click="sendSmsCode"
      >
        {{ smsCountdown > 0 ? `${smsCountdown}s` : '获取验证码' }}
      </el-button>
    </div>
  </el-form-item>
</template>

<!-- 邮箱登录 -->
<template v-else-if="activeTab === 'EMAIL'">
  <el-form-item prop="identifier">
    <el-input
      v-model="loginForm.identifier"
      placeholder="请输入邮箱"
      :prefix-icon="Message"
      clearable
    />
  </el-form-item>

  <el-form-item prop="credential">
    <div class="captcha-row">
      <el-input
        v-model="loginForm.credential"
        placeholder="请输入验证码"
        :prefix-icon="Picture"
        clearable
        class="captcha-input"
        maxlength="6"
      />
      <el-button
        :disabled="emailCountdown > 0"
        class="send-btn"
        @click="sendEmailCode"
      >
        {{ emailCountdown > 0 ? `${emailCountdown}s` : '获取验证码' }}
      </el-button>
    </div>
  </el-form-item>
</template>

<el-form-item>
  <el-button
    type="primary"
    :loading="loading"
    class="login-btn"
    @click="handleLogin"
  >

```

```

    {{ loading ? '登录中...' : '登 录' }}
  </el-button>
</el-form-item>

<el-form-item>
  <el-button
    class="reset-btn"
    @click="resetForm"
  >
    重 置
  </el-button>
</el-form-item>
</el-form>

<div class="login-tips">
  <span class="tips-label">默认账号: </span>
  <span class="tips-value">admin / admin123</span>
</div>
</div>

<div class="login-footer">
  <p class="company-name">
    河南爱编程网络科技有限公司
  </p>
  <a
    href="https://beian.miit.gov.cn/"
    target="_blank"
    rel="noopener noreferrer"
    class="icp-link"
  >
    豫ICP备2024074107号-2
  </a>
</div>
</div>
</div>
</template>

<style scoped>
.login-container {
  display: flex;
  min-height: 100vh;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

/* 左侧品牌区域 */
.login-brand {
  flex: 1;
  display: none;
  position: relative;
  overflow: hidden;

```

```
background: linear-gradient(135deg, rgba(37, 99, 235, 0.9) 0%, rgba(124, 58, 237, 0.9) 100%);
color: white;
}

@media (min-width: 1024px) {
  .login-brand {
    display: flex;
    align-items: center;
    justify-content: center;
  }
}

.brand-content {
  position: relative;
  z-index: 10;
  text-align: center;
  padding: 3rem;
}

.brand-logo {
  margin-bottom: 2rem;
}

.logo-img {
  width: 80px;
  height: 80px;
  object-fit: contain;
  filter: brightness(0) invert(1);
}

.brand-title {
  font-size: 2rem;
  font-weight: 700;
  margin-bottom: 0.75rem;
  letter-spacing: 0.025em;
}

.brand-subtitle {
  font-size: 1.125rem;
  opacity: 0.9;
  margin-bottom: 3rem;
}

.brand-features {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.feature-item {
```

```
display: flex;
align-items: center;
justify-content: center;
gap: 0.75rem;
font-size: 1rem;
}

.feature-icon {
width: 24px;
height: 24px;
opacity: 0.9;
}

.brand-decoration {
position: absolute;
inset: 0;
overflow: hidden;
pointer-events: none;
}

.decoration-circle {
position: absolute;
border-radius: 50%;
background: rgba(255, 255, 255, 0.1);
}

.decoration-circle-1 {
width: 300px;
height: 300px;
top: -100px;
left: -100px;
}

.decoration-circle-2 {
width: 200px;
height: 200px;
bottom: -50px;
right: -50px;
}

.decoration-circle-3 {
width: 150px;
height: 150px;
top: 50%;
right: 10%;
}

/* 右侧登录区域 */
.login-form-wrapper {
flex: 1;
}
```

```
display: flex;
flex-direction: column;
justify-content: center;
align-items: center;
padding: 2rem;
background: rgba(255, 255, 255, 0.98);
backdrop-filter: blur(10px);
}

@media (min-width: 1024px) {
  .login-form-wrapper {
    max-width: 500px;
    flex: none;
  }
}

.login-form-container {
  width: 100%;
  max-width: 380px;
}

.mobile-logo {
  display: flex;
  justify-content: center;
  margin-bottom: 1.5rem;
}

@media (min-width: 1024px) {
  .mobile-logo {
    display: none;
  }
}

.logo-img-mobile {
  width: 60px;
  height: 60px;
  object-fit: contain;
}

.login-header {
  text-align: center;
  margin-bottom: 2rem;
}

.login-title {
  font-size: 1.75rem;
  font-weight: 600;
  color: #1e293b;
  margin-bottom: 0.5rem;
}
```

```
.login-subtitle {
  font-size: 0.9375rem;
  color: #64748b;
}

/* 登录方式 Tab */
.login-tabs {
  display: flex;
  gap: 0.5rem;
  margin-bottom: 1.5rem;
}

.tab-btn {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
  padding: 0.75rem;
  border: 1px solid #e2e8f0;
  background: #fff;
  border-radius: 8px;
  font-size: 0.875rem;
  color: #64748b;
  cursor: pointer;
  transition: all 0.2s ease;
}

.tab-btn:hover {
  border-color: #2563eb;
  color: #2563eb;
}

.tab-btn.active {
  background: #2563eb;
  border-color: #2563eb;
  color: #fff;
}

.login-form {
  margin-bottom: 1.5rem;
}

.captcha-row {
  display: flex;
  width: 100%;
  gap: 0.75rem;
}
```

后30页（第31页-第60页）

```
=====
文件: backend/src/main/java/com/devlovecode/aiperm/AipermApplication.java
=====
```

```
package com.devlovecode.aiperm;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.modulith.Modulith;
import org.springframework.scheduling.annotation.EnableAsync;

/**
 * @author shichenyang
 */
@EnableAsync
@SpringBootApplication
@Modulith(systemName = "AIPerm", additionalPackages = "com.devlovecode.aiperm.modules")
public class AipermApplication {

    public static void main(String[] args) {
        SpringApplication.run(AipermApplication.class, args);
    }

}
```

```
=====
文件: backend/src/main/java/com/devlovecode/aiperm/common/domain/BaseEntity.java
=====
```

```
package com.devlovecode.aiperm.common.domain;

import jakarta.persistence.*;
import lombok.Data;
import org.hibernate.annotations.SQLRestriction;

import java.io.Serializable;
import java.io.Serializable;
import java.time.LocalDateTime;

/**
 * 基础实体类 所有实体类的父类，包含通用字段
 */
@Data
@MappedSuperclass
@SQLRestriction("deleted = 0")
public class BaseEntity implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;
}
```

```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private LocalDateTime createTime;

    private LocalDateTime updateTime;

    private String createBy;

    private String updateBy;

    private Integer deleted;

    @Version
    private Integer version;

    @PrePersist
    public void prePersist() {
        if (deleted == null) {
            deleted = 0;
        }
    }
}

```

```

=====
文件: backend/src/main/java/com/devlovecode/aiperm/common/domain/R.java
=====

```

```

package com.devlovecode.aiperm.common.domain;

```

```

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

```

```

import java.io.Serializable;
import java.io.Serializableizable;

```

```

/**
 * 统一返回结果封装
 *
 * @param <T> 数据类型
 * @author devlovecode
 */
@Data
@NoArgsConstructor
@AllArgsConstructor

```

```
@Schema(description = "统一返回结果")
public class R<T> implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;

    @Schema(description = "状态码")
    private Integer code;

    @Schema(description = "返回消息")
    private String message;

    @Schema(description = "返回数据")
    private T data;

    /**
     * 成功返回（无数据）
     */
    public static <T> R<T> ok() {
        return new R<>(200, "操作成功", null);
    }

    /**
     * 成功返回（带数据）
     */
    public static <T> R<T> ok(T data) {
        return new R<>(200, "操作成功", data);
    }

    /**
     * 成功返回（自定义消息和数据）
     */
    public static <T> R<T> ok(String message, T data) {
        return new R<>(200, message, data);
    }

    /**
     * 失败返回（默认消息）
     */
    public static <T> R<T> fail() {
        return new R<>(500, "操作失败", null);
    }

    /**
     * 失败返回（自定义消息）
     */
    public static <T> R<T> fail(String message) {
        return new R<>(500, message, null);
    }
}
```

```

/**
 * 失败返回（自定义状态码和消息）
 */
public static <T> R<T> fail(Integer code, String message) {
    return new R<>(code, message, null);
}

/**
 * 失败返回（使用错误码枚举）
 */
public static <T> R<T> fail(com.devlovecode.aiperm.common.enums.ErrorCode errorCode) {
    return new R<>(errorCode.getCode(), errorCode.getMessage(), null);
}

/**
 * 判断是否成功
 */
public boolean isSuccess() {
    return this.code != null && this.code == 200;
}
}

```

```

=====
文件: backend/src/main/java/com/devlovecode/aiperm/common/domain/PageResult.java
=====

```

```

package com.devlovecode.aiperm.common.domain;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.domain.Page;

import java.io.Serializable;
import java.io.Serializable;
import java.util.Collections;
import java.util.List;
import java.util.function.Function;
import java.util.stream.Collectors;

/**
 * 分页结果封装
 *
 * @param <T> 数据类型
 * @author devlovecode
 */
@Data
@NoArgsConstructor

```

```

@AllArgsConstructor
@Schema(description = "分页结果")
public class PageResult<T> implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;

    @Schema(description = "总记录数")
    private Long total;

    @Schema(description = "数据列表")
    private List<T> list;

    @Schema(description = "当前页码")
    private Long pageNum;

    @Schema(description = "每页条数")
    private Long pageSize;

    @Schema(description = "总页数")
    private Long pages;

    /**
     * 构建分页结果
     */
    public static <T> PageResult<T> of(Long total, List<T> list, Long pageNum, Long pageSize) {
        PageResult<T> result = new PageResult<>();
        result.setTotal(total);
        result.setList(list);
        result.setPageNum(pageNum);
        result.setPageSize(pageSize);
        result.setPages(pageSize > 0 ? (total + pageSize - 1) / pageSize : 0L);
        return result;
    }

    /**
     * 构建空分页结果
     */
    public static <T> PageResult<T> empty(Long pageNum, Long pageSize) {
        return of(0L, Collections.emptyList(), pageNum, pageSize);
    }

    /**
     * 从 Spring Data JPA Page 转换（JPA 页码从 0 开始，前端从 1 开始）
     */
    public static <T> PageResult<T> fromJpaPage(Page<T> page) {
        return of(page.getTotalElements(), page.getContent(), (long) page.getNumber() + 1, (long) page.getSize());
    }

    /**

```

```

    * 转换列表元素类型
    */
    public <R> PageResult<R> map(Function<T, R> mapper) {
        List<R> mappedList = this.list.stream().map(mapper).collect(Collectors.toList());
        return of(this.total, mappedList, this.pageNum, this.pageSize);
    }
}

```

```

=====
文件: backend/src/main/java/com/devlovecode/aiperm/common/domain/Views.java
=====

```

```

package com.devlovecode.aiperm.common.domain;

```

```

/**
 * Jackson 视图定义，用于 DTO 多场景复用
 */
public class Views {

    /** 创建场景 */
    public interface Create {

    }

    /** 更新场景 */
    public interface Update {

    }

    /** 查询场景 */
    public interface Query {

    }
}

```

```

=====
文件: backend/src/main/java/com/devlovecode/aiperm/common/annotation/Log.java
=====

```

```

package com.devlovecode.aiperm.common.annotation;

```

```

import com.devlovecode.aiperm.common.enums.OperType;

```

```

import java.lang.annotation.*;

```

```

/**
 * @author shichenyang
 */

```

```

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Log {

    /** 操作模块名称，如"用户管理" */
    String title() default "";

    /** 操作类型 */
    OperType operType() default OperType.OTHER;

    /** 是否保存请求参数，默认 true */
    boolean saveRequestParam() default true;

    /** 是否保存响应结果，默认 false（避免响应体过大） */
    boolean saveResponseResult() default false;

}

```

文件: backend/src/main/java/com/devlovecode/aiperm/common/exception/BusinessException.java

```

package com.devlovecode.aiperm.common.exception;

import com.devlovecode.aiperm.common.enums.ErrorCode;
import lombok.Getter;

import java.io.Serial;

/**
 * 业务异常
 *
 * @author DevLoveCode
 */
@Getter
public class BusinessException extends RuntimeException {

    @Serial
    private static final long serialVersionUID = 1L;

    /**
     * 错误码
     */
    private final Integer code;

    /**
     * 错误信息
     */
    private final String message;
}

```

```

public BusinessException(String message) {
    super(message);
    this.code = 500;
    this.message = message;
}

public BusinessException(Integer code, String message) {
    super(message);
    this.code = code;
    this.message = message;
}

public BusinessException(ErrorCode errorCode) {
    super(errorCode.getMessage());
    this.code = errorCode.getCode();
    this.message = errorCode.getMessage();
}

public BusinessException(ErrorCode errorCode, String message) {
    super(message);
    this.code = errorCode.getCode();
    this.message = message;
}
}

=====
文件: backend/src/main/java/com/devlovecode/aiperm/common/repository/BaseJpaRepository.java
=====

package com.devlovecode.aiperm.common.repository;

import com.devlovecode.aiperm.common.domain.BaseEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.NoRepositoryBean;
import org.springframework.data.repository.query.Param;

import java.time.LocalDateTime;
import java.util.List;

/**
 * JPA Repository 基础接口 提供软删除和批量软删除，禁止硬删除
 *
 * @param <T> 实体类型，必须继承 BaseEntity
 */
@NoRepositoryBean

```

```

public interface BaseJpaRepository<T extends BaseEntity> extends JpaRepository<T, Long>, JpaSpecificationExecutor<T> {

    /**
     * 软删除
     */
    @Modifying
    @Query("UPDATE #{entityName} e SET e.deleted = 1, e.updateTime = :updateTime WHERE e.id = :id")
    int softDelete(@Param("id") Long id, @Param("updateTime") LocalDateTime updateTime);

    /**
     * 批量软删除
     */
    @Modifying
    @Query("UPDATE #{entityName} e SET e.deleted = 1, e.updateTime = :updateTime WHERE e.id IN :ids")
    int softDeleteByIds(@Param("ids") List<Long> ids, @Param("updateTime") LocalDateTime updateTime);

    /**
     * 禁止硬删除，强制使用软删除
     */
    @Override
    default void deleteById(Long id) {
        throw new UnsupportedOperationException("请使用 softDelete() 进行软删除");
    }

    @Override
    default void delete(T entity) {
        throw new UnsupportedOperationException("请使用 softDelete() 进行软删除");
    }

    @Override
    default void deleteAllById(Iterable<? extends Long> ids) {
        throw new UnsupportedOperationException("请使用 softDeleteByIds() 进行软删除");
    }

    @Override
    default void deleteAll(Iterable<? extends T> entities) {
        throw new UnsupportedOperationException("请使用 softDeleteByIds() 进行软删除");
    }

    @Override
    default void deleteAll() {
        throw new UnsupportedOperationException("请使用 softDeleteByIds() 进行软删除");
    }
}

=====
文件: backend/src/main/java/com/devlovecode/aiperm/common/repository/SpecificationUtils.java
=====

```

```

package com.devlovecode.aiperm.common.repository;

import jakarta.persistence.criteria.Predicate;
import org.springframework.data.jpa.domain.Specification;

import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

/**
 * Specification 工具类 替代 SqlBuilder， 提供动态查询条件构建
 */
public final class SpecificationUtils {

    private SpecificationUtils() {
    }

    /**
     * LIKE 模糊查询
     */
    public static <T> Specification<T> like(String fieldName, String value) {
        if (value == null || value.isBlank()) {
            return null;
        }
        return (root, query, cb) -> cb.like(root.get(fieldName), "%" + value + "%");
    }

    /**
     * 精确匹配
     */
    public static <T> Specification<T> eq(String fieldName, Object value) {
        if (value == null) {
            return null;
        }
        return (root, query, cb) -> cb.equal(root.get(fieldName), value);
    }

    /**
     * 大于等于
     */
    public static <T, V extends Comparable<? super V>> Specification<T> ge(String fieldName, V value) {
        if (value == null) {
            return null;
        }
        return (root, query, cb) -> cb.greaterThanOrEqualTo(root.get(fieldName), value);
    }

    /**
     * 小于等于
     */

```

```

public static <T, V extends Comparable<? super V>> Specification<T> le(String fieldName, V value) {
    if (value == null) {
        return null;
    }
    return (root, query, cb) -> cb.lessThanOrEqualTo(root.get(fieldName), value);
}

/**
 * IN 查询
 */
public static <T> Specification<T> in(String fieldName, Collection<?> values) {
    if (values == null || values.isEmpty()) {
        return null;
    }
    return (root, query, cb) -> root.get(fieldName).in(values);
}

/**
 * 组合多个条件 (AND)
 */
@SafeVarargs
public static <T> Specification<T> and(Specification<T>... specs) {
    return (root, query, cb) -> {
        List<Predicate> predicates = new ArrayList<>();
        for (Specification<T> spec : specs) {
            if (spec != null) {
                Predicate p = spec.toPredicate(root, query, cb);
                if (p != null) {
                    predicates.add(p);
                }
            }
        }
        return predicates.isEmpty() ? null : cb.and(predicates.toArray(new Predicate[0]));
    };
}
}

```

```

=====
文件: backend/src/main/java/com/devlovecode/aiperm/config/SaTokenConfig.java
=====

```

```

package com.devlovecode.aiperm.config;

import cn.dev33.satoken.interceptor.SaInterceptor;
import cn.dev33.satoken.stp.StpUtil;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

```

```

/**
 * Sa-Token配置
 *
 * @author DevLoveCode
 */
@Configuration
public class SaTokenConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        // 注册Sa-Token拦截器，校验规则为StpUtil.checkLogin()登录校验
        registry.addInterceptor(new SaInterceptor(handle -> StpUtil.checkLogin()))
            .addPathPatterns("/**")
            .excludePathPatterns(
                // 排除认证相关接口
                "/auth/login", "/auth/unified-login", "/auth/captcha", "/auth/login-config",
                // 排除验证码发送接口
                "/captcha/**",
                // 排除 OAuth 登录接口
                "/oauth/login/**",
                // 审批回调接口（来自企业IM服务端）
                "/approval/callback/**",
                // 排除Swagger相关
                "/swagger-ui/**", "/swagger-resources/**", "/v3/api-docs/**", "/webjars/**", "/doc.html",
                // 排除静态资源
                "/favicon.ico", "/error")
            .order(0);
    }
}

```

文件: backend/src/main/java/com/devlovecode/aiperm/config/StpInterfaceImpl.java

```

package com.devlovecode.aiperm.config;

import cn.dev33.satoken.stp.StpInterface;
import com.devlovecode.aiperm.modules.auth.core.service.AuthService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

import java.util.List;

/**
 * Sa-Token 权限认证实现 实现 StpInterface 接口，提供用户的角色和权限数据
 */
@Component
@RequiredArgsConstructor
public class StpInterfaceImpl implements StpInterface {

```

```

private final AuthService authService;

/**
 * 返回一个账号所拥有的权限码集合
 */
@Override
public List<String> getPermissionList(Object loginId, String loginType) {
    Long userId = Long.parseLong(loginId.toString());

    if (authService.isSuperAdmin(userId)) {
        // 超级管理员返回所有权限
        return authService.getAllPermissions();
    }

    return authService.getUserPermissions(userId);
}

/**
 * 返回一个账号所拥有的角色标识集合
 */
@Override
public List<String> getRoleList(Object loginId, String loginType) {
    Long userId = Long.parseLong(loginId.toString());
    if (authService.isSuperAdmin(userId)) {
        return List.of("super_admin");
    }
    // 统一从数据库获取角色
    return authService.getUserRoles(userId);
}
}

=====
文件: backend/src/main/java/com/devlovecode/aiperm/modules/system/rbac/controller/SysUserController.java
=====

package com.devlovecode.aiperm.modules.system.rbac.controller;

import cn.dev33.satoken.annotation.SaCheckLogin;
import cn.dev33.satoken.annotation.SaCheckPermission;
import com.devlovecode.aiperm.common.annotation.Log;
import com.devlovecode.aiperm.common.domain.PageResult;
import com.devlovecode.aiperm.common.domain.R;
import com.devlovecode.aiperm.common.domain.Views;
import com.devlovecode.aiperm.common.enums.OperType;
import com.devlovecode.aiperm.common.vo.ImportResultVO;
import com.devlovecode.aiperm.modules.system.rbac.dto.UserDTO;
import com.devlovecode.aiperm.modules.system.rbac.service.UserService;
import com.devlovecode.aiperm.modules.system.rbac.vo.UserVO;

```

```

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.groups.Default;
import lombok.RequiredArgsConstructor;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.multipart.MultipartFile;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@Tag(name = "用户管理")
@RestController
@RequestMapping("/system/user")
@SaCheckLogin
@RequiredArgsConstructor
public class SysUserController {

    private final UserService userService;

    @Operation(summary = "分页查询用户列表")
    @SaCheckPermission("system:user:list")
    @Log(title = "用户管理", operType = OperType.QUERY)
    @GetMapping
    public R<PageResult<UserVO>> page(@Validated({ Default.class, Views.Query.class }) UserDTO dto) {
        return R.ok(userService.queryPage(dto));
    }

    @Operation(summary = "导出用户")
    @SaCheckPermission("system:user:export")
    @Log(title = "用户管理", operType = OperType.EXPORT)
    @GetMapping("/export")
    public void export(@Validated({ Default.class, Views.Query.class }) UserDTO dto, HttpServletResponse response) {
        userService.export(dto, response);
    }

    @Operation(summary = "下载用户导入模板")
    @SaCheckPermission("system:user:import")
    @GetMapping("/import-template")
    public void importTemplate(HttpServletResponse response) {
        userService.downloadImportTemplate(response);
    }

    @Operation(summary = "导入用户")
    @SaCheckPermission("system:user:import")
    @Log(title = "用户管理", operType = OperType.IMPORT)
    @PostMapping("/import")
    public R<ImportResultVO> importUsers(@RequestParam("file") MultipartFile file) {
        return R.ok(userService.importUsers(file));
    }
}

```

```

@Operation(summary = "根据ID查询用户")
@SaCheckPermission("system:user:list")
@GetMapping("/{id}")
public R<UserVO> getById(@PathVariable Long id) {
    return R.ok(userService.findById(id));
}

@Operation(summary = "创建用户")
@SaCheckPermission("system:user:create")
@Log(title = "用户管理", operType = OperType.CREATE)
@PostMapping
public R<Void> create(@RequestBody @Validated({ Default.class, Views.Create.class }) UserDTO dto) {
    userService.create(dto);
    return R.ok();
}

@Operation(summary = "更新用户")
@SaCheckPermission("system:user:update")
@Log(title = "用户管理", operType = OperType.UPDATE)
@PutMapping("/{id}")
public R<Void> update(@PathVariable Long id,
    @RequestBody @Validated({ Default.class, Views.Update.class }) UserDTO dto) {
    userService.update(id, dto);
    return R.ok();
}

@Operation(summary = "删除用户")
@SaCheckPermission("system:user:delete")
@Log(title = "用户管理", operType = OperType.DELETE)
@DeleteMapping("/{id}")
public R<Void> delete(@PathVariable Long id) {
    userService.delete(id);
    return R.ok();
}

@Operation(summary = "批量删除用户")
@SaCheckPermission("system:user:delete")
@Log(title = "用户管理", operType = OperType.DELETE)
@DeleteMapping("/batch")
public R<Void> deleteBatch(@RequestBody List<Long> ids) {
    userService.deleteBatch(ids);
    return R.ok();
}

@Operation(summary = "重置用户密码")
@SaCheckPermission("system:user:update")
@Log(title = "用户管理", operType = OperType.UPDATE)
@PutMapping("/{id}/reset-password")
public R<Void> resetPassword(@PathVariable Long id, @RequestBody UserDTO dto) {

```

```

        userService.resetPassword(id, dto.getNewPassword());
        return R.ok();
    }

    @Operation(summary = "修改用户状态")
    @SaCheckPermission("system:user:update")
    @Log(title = "用户管理", operType = OperType.UPDATE)
    @PutMapping("/{id}/status")
    public R<Void> changeStatus(@PathVariable Long id, @RequestParam(name = "status") Integer status) {
        userService.changeStatus(id, status);
        return R.ok();
    }
}

}

=====
文件: backend/src/main/java/com/devlovecode/aiperm/modules/system/rbac/controller/SysRoleController.java
=====

package com.devlovecode.aiperm.modules.system.rbac.controller;

import cn.dev33.satoken.annotation.SaCheckLogin;
import cn.dev33.satoken.annotation.SaCheckPermission;
import com.devlovecode.aiperm.common.annotation.Log;
import com.devlovecode.aiperm.common.domain.PageResult;
import com.devlovecode.aiperm.common.domain.R;
import com.devlovecode.aiperm.common.domain.Views;
import com.devlovecode.aiperm.common.enums.OperType;
import com.devlovecode.aiperm.modules.system.rbac.dto.RoleDTO;
import com.devlovecode.aiperm.modules.system.rbac.entity.SysRole;
import com.devlovecode.aiperm.modules.system.rbac.service.RoleService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.groups.Default;
import lombok.RequiredArgsConstructor;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@Tag(name = "角色管理")
@RestController
@RequestMapping("/system/role")
@SaCheckLogin
@RequiredArgsConstructor
public class SysRoleController {

    private final RoleService roleService;

    @Operation(summary = "分页查询角色列表")

```

```

@SaCheckPermission("system:role:list")
@Log(title = "角色管理", operType = OperType.QUERY)
@GetMapping
public R<PageResult<SysRole>> page(@Validated({ Default.class, Views.Query.class }) RoleDTO dto) {
    return R.ok(roleService.queryPage(dto));
}

@Operation(summary = "查询所有角色")
@SaCheckPermission("system:role:list")
@GetMapping("/{all}")
public R<List<SysRole>> list() {
    return R.ok(roleService.listAll());
}

@Operation(summary = "根据ID查询角色")
@SaCheckPermission("system:role:list")
@GetMapping("/{id}")
public R<SysRole> getById(@PathVariable Long id) {
    return R.ok(roleService.findById(id));
}

@Operation(summary = "获取角色的菜单ID列表")
@SaCheckPermission("system:role:list")
@GetMapping("/{id}/menus")
public R<List<Long>> getRoleMenus(@PathVariable Long id) {
    return R.ok(roleService.getMenuIds(id));
}

@Operation(summary = "创建角色")
@SaCheckPermission("system:role:create")
@Log(title = "角色管理", operType = OperType.CREATE)
@PostMapping
public R<Void> create(@RequestBody @Validated({ Default.class, Views.Create.class }) RoleDTO dto) {
    roleService.create(dto);
    return R.ok();
}

@Operation(summary = "更新角色")
@SaCheckPermission("system:role:update")
@Log(title = "角色管理", operType = OperType.UPDATE)
@PutMapping("/{id}")
public R<Void> update(@PathVariable Long id,
    @RequestBody @Validated({ Default.class, Views.Update.class }) RoleDTO dto) {
    roleService.update(id, dto);
    return R.ok();
}

@Operation(summary = "删除角色")
@SaCheckPermission("system:role:delete")
@Log(title = "角色管理", operType = OperType.DELETE)

```

```

    @DeleteMapping("/{id}")
    public R<Void> delete(@PathVariable Long id) {
        roleService.delete(id);
        return R.ok();
    }

    @Operation(summary = "分配角色菜单")
    @SaCheckPermission("system:role:update")
    @Log(title = "角色管理", operType = OperType.UPDATE)
    @PostMapping("/{id}/menus")
    public R<Void> assignMenus(@PathVariable Long id, @RequestBody List<Long> menuIds) {
        roleService.assignMenus(id, menuIds);
        return R.ok();
    }
}

=====
文件: backend/src/main/java/com/devlovecode/aiperm/modules/system/rbac/controller/SysMenuController.java
=====

package com.devlovecode.aiperm.modules.system.rbac.controller;

import cn.dev33.satoken.annotation.SaCheckLogin;
import cn.dev33.satoken.annotation.SaCheckPermission;
import com.devlovecode.aiperm.common.annotation.Log;
import com.devlovecode.aiperm.common.domain.R;
import com.devlovecode.aiperm.common.domain.Views;
import com.devlovecode.aiperm.common.enums.OperType;
import com.devlovecode.aiperm.modules.system.rbac.dto.MenuDTO;
import com.devlovecode.aiperm.modules.system.rbac.entity.SysMenu;
import com.devlovecode.aiperm.modules.system.rbac.service.MenuService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.groups.Default;
import lombok.RequiredArgsConstructor;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@Tag(name = "菜单管理")
@RestController
@RequestMapping("/system/menu")
@SaCheckLogin
@RequiredArgsConstructor
public class SysMenuController {

    private final MenuService menuService;

```

```

@Operation(summary = "查询菜单树")
@SaCheckPermission("system:menu:list")
@GetMapping("/tree")
public R<List<SysMenu>> tree() {
    return R.ok(menuService.getMenuTree());
}

@Operation(summary = "查询所有菜单")
@SaCheckPermission("system:menu:list")
@GetMapping
public R<List<SysMenu>> list() {
    return R.ok(menuService.listAll());
}

@Operation(summary = "根据ID查询菜单")
@SaCheckPermission("system:menu:list")
@GetMapping("/{id}")
public R<SysMenu> getById(@PathVariable Long id) {
    return R.ok(menuService.findById(id));
}

@Operation(summary = "查询子菜单列表")
@SaCheckPermission("system:menu:list")
@GetMapping("/children/{parentId}")
public R<List<SysMenu>> getChildren(@PathVariable Long parentId) {
    return R.ok(menuService.listByParentId(parentId));
}

@Operation(summary = "创建菜单")
@SaCheckPermission("system:menu:create")
@Log(title = "菜单管理", operType = OperType.CREATE)
@PostMapping
public R<Void> create(@RequestBody @Validated({ Default.class, Views.Create.class }) MenuDTO dto) {
    menuService.create(dto);
    return R.ok();
}

@Operation(summary = "更新菜单")
@SaCheckPermission("system:menu:update")
@Log(title = "菜单管理", operType = OperType.UPDATE)
@PutMapping("/{id}")
public R<Void> update(@PathVariable Long id,
    @RequestBody @Validated({ Default.class, Views.Update.class }) MenuDTO dto) {
    menuService.update(id, dto);
    return R.ok();
}

@Operation(summary = "删除菜单")
@SaCheckPermission("system:menu:delete")
@Log(title = "菜单管理", operType = OperType.DELETE)

```

```
@DeleteMapping("/{id}")
public R<Void> delete(@PathVariable Long id) {
    menuService.delete(id);
    return R.ok();
}

}
```

=====

文件: backend/src/main/java/com/devlovecode/aiperm/modules/auth/core/service/AuthService.java

=====

```
package com.devlovecode.aiperm.modules.auth.core.service;

import cn.dev33.satoken.stp.StpUtil;
import cn.hutool.captcha.CaptchaUtil;
import cn.hutool.captcha.LineCaptcha;
import cn.hutool.core.lang.UUID;
import cn.hutool.crypto.digest.BCrypt;
import com.devlovecode.aiperm.common.exception.BusinessException;
import com.devlovecode.aiperm.common.util.ClientIpUtils;
import com.devlovecode.aiperm.modules.audit.api.LoginAuditApi;
import com.devlovecode.aiperm.modules.auth.core.dto.UnifiedLoginDTO;
import com.devlovecode.aiperm.modules.auth.core.dto.request.LoginRequest;
import com.devlovecode.aiperm.modules.auth.core.enums.LoginType;
import com.devlovecode.aiperm.modules.auth.core.strategy.LoginStrategy;
import com.devlovecode.aiperm.modules.auth.core.strategy.LoginStrategyFactory;
import com.devlovecode.aiperm.modules.auth.core.vo.CaptchaVO;
import com.devlovecode.aiperm.modules.auth.core.vo.LoginConfigVO;
import com.devlovecode.aiperm.modules.auth.core.vo.LoginVO;
import com.devlovecode.aiperm.modules.auth.core.vo.MenuVO;
import com.devlovecode.aiperm.modules.auth.core.vo.UserInfoVO;
import com.devlovecode.aiperm.modules.monitor.api.OnlineSessionApi;
import com.devlovecode.aiperm.modules.system.api.SystemAccess;
import com.devlovecode.aiperm.modules.system.api.SystemMenuDescriptor;
import com.devlovecode.aiperm.modules.system.api.SystemUserAccount;
import jakarta.servlet.http.HttpServletRequest;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.http.HttpHeaders;
import org.springframework.stereotype.Service;
import org.springframework.web.context.request.RequestAttributes;
import org.springframework.web.context.request.RequestContextHolder;
import org.springframework.web.context.request.ServletRequestAttributes;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
```

```

import java.util.Map;
import java.util.concurrent.TimeUnit;
import java.util.stream.Collectors;

/**
 * 认证服务
 *
 * @author DevLoveCode
 */
@Service
@Slf4j
@RequiredArgsConstructor
public class AuthService {

    private final StringRedisTemplate redisTemplate;

    private final LoginStrategyFactory loginStrategyFactory;

    private final SystemAccess systemAccess;

    private final LoginAuditApi loginAuditApi;

    private final OnlineSessionApi onlineSessionApi;

    @Value("${auth.captcha.enabled:true}")
    private boolean captchaEnabled;

    private static final String CAPTCHA_PREFIX = "captcha:";

    private static final long CAPTCHA_EXPIRE = 5;

    /**
     * 生成验证码
     */
    public CaptchaVO generateCaptcha() {
        LineCaptcha captcha = CaptchaUtil.createLineCaptcha(120, 40, 4, 20);
        String code = captcha.getCode();
        String imageBase64 = captcha.getImageBase64Data();

        String captchaKey = UUID.fastUUID().toString(true);
        redisTemplate.opsForValue().set(CAPTCHA_PREFIX + captchaKey, code.toLowerCase(), CAPTCHA_EXPIRE, TimeUnit.MINUTES);

        return CaptchaVO.builder().captchaKey(captchaKey).captchaImage(imageBase64).build();
    }

    /**
     * 登录
     */
    public LoginVO login(LoginRequest request) {
        HttpServletRequest currentRequest = getCurrentRequest();

```

```

String ip = ClientIpUtils.getCurrentRequestIp();
String userAgent = resolveUserAgent(currentRequest);
try {
    validateCaptcha(request.getCaptchaKey(), request.getCaptcha());

    SystemUserAccount user = systemAccess.findUserByUsername(request.getUsername())
        .orElseThrow(() -> new BusinessException("用户名或密码错误"));

    if (user.status() != null && user.status() == 0) {
        throw new BusinessException("账号已被禁用");
    }

    if (!BCrypt.checkpw(request.getPassword(), user.password())) {
        throw new BusinessException("用户名或密码错误");
    }

    StpUtil.login(user.id());
    systemAccess.updateLoginInfo(user.id(), ip, LocalDateTime.now());
    loginAuditApi.recordSuccess(user.id(), user.username(), ip, userAgent, currentRequest);
    onlineSessionApi.registerLoginSession(user.id(), user.username(), ip, userAgent);

    return LoginVO.builder().token(StpUtil.getTokenValue()).userInfo(buildUserInfo(user)).build();
}
catch (BusinessException e) {
    loginAuditApi.recordFailed(request.getUsername(), ip, e.getMessage(), userAgent, currentRequest);
    throw e;
}
catch (Exception e) {
    log.error("传统登录异常: username={}, ip={}", request.getUsername(), ip, e);
    loginAuditApi.recordFailed(request.getUsername(), ip, "系统异常", userAgent, currentRequest);
    throw e;
}
}

/**
 * 统一登录（支持多种登录方式）
 */
public LoginVO unifiedLogin(UnifiedLoginDTO dto, HttpServletRequest request) {
    String ip = ClientIpUtils.getClientIp(request);
    String userAgent = resolveUserAgent(request);

    if (LoginType.PASSWORD.getCode().equalsIgnoreCase(dto.getLoginType())) {
        validateCaptcha(dto.getImageCaptchaKey(), dto.getImageCaptcha());
    }

    try {
        LoginStrategy strategy = loginStrategyFactory.getStrategy(dto.getLoginType());
        return strategy.login(dto.getIdentifier(), dto.getCredential(), ip, userAgent, request);
    }
    catch (BusinessException e) {

```

```

        loginAuditApi.recordFailed(dto.getIdentifier(), ip, e.getMessage(), userAgent, request);
        throw e;
    }
    catch (Exception e) {
        log.error("统一登录异常: loginType={}, identifier={}, ip={}", dto.getLoginType(), dto.getIdentifier(), ip, e);
        loginAuditApi.recordFailed(dto.getIdentifier(), ip, "系统异常", userAgent, request);
        throw e;
    }
}

/**
 * 获取登录配置
 */
public LoginConfigVO getLoginConfig() {
    boolean smsEnabled = systemAccess.getBooleanConfig("login.sms.enabled", false);
    boolean emailEnabled = systemAccess.getBooleanConfig("login.email.enabled", false);

    List<LoginConfigVO.OAuthConfig> oauthConfigs = new ArrayList<>();
    if (systemAccess.getBooleanConfig("oauth.wework.enabled", false)) {
        oauthConfigs.add(LoginConfigVO.OAuthConfig.builder()
            .platform("WEWORK")
            .displayName("企业微信")
            .icon("/icons/wework.svg")
            .enabled(true)
            .build());
    }
    if (systemAccess.getBooleanConfig("oauth.dingtalk.enabled", false)) {
        oauthConfigs.add(LoginConfigVO.OAuthConfig.builder()
            .platform("DINGTALK")
            .displayName("钉钉")
            .icon("/icons/dingtalk.svg")
            .enabled(true)
            .build());
    }
    if (systemAccess.getBooleanConfig("oauth.feishu.enabled", false)) {
        oauthConfigs.add(LoginConfigVO.OAuthConfig.builder()
            .platform("FEISHU")
            .displayName("飞书")
            .icon("/icons/feishu.svg")
            .enabled(true)
            .build());
    }

    return LoginConfigVO.builder()
        .passwordEnabled(true)
        .smsEnabled(smsEnabled)
        .emailEnabled(emailEnabled)
        .oauthConfigs(oauthConfigs)
        .build();
}

```

```

/**
 * 登出
 */
public void logout() {
    onlineSessionApi.logoutCurrentToken();
    StpUtil.logout();
}

/**
 * 获取当前用户信息
 */
public LoginVO.UserInfo getCurrentUserInfo() {
    Long userId = StpUtil.getLoginIdAsLong();
    SystemUserAccount user = systemAccess.findUserById(userId).orElseThrow(() -> new BusinessException("用户不存在"));
    return buildUserInfo(user);
}

/**
 * 获取当前用户完整信息（包含角色和权限）
 */
public UserInfoVO getUserInfo() {
    Long userId = StpUtil.getLoginIdAsLong();
    SystemUserAccount user = systemAccess.findUserById(userId).orElseThrow(() -> new BusinessException("用户不存在"));

    List<String> roles;
    List<String> permissions;

    if (isSuperAdmin(userId)) {
        roles = List.of("super_admin");
        permissions = getAllPermissions();
    }
    else {
        roles = getUserRoles(userId);
        permissions = getUserPermissions(userId);
    }

    return UserInfoVO.builder()
        .id(user.id())
        .username(user.username())
        .nickname(user.nickname())
        .avatar(user.avatar())
        .roles(roles)
        .permissions(permissions)
        .build();
}

/**
 * 获取当前用户可访问的菜单
 */

```

```

public List<MenuVO> getUserMenus() {
    Long userId = StpUtil.getLoginIdAsLong();
    return buildMenuTree(systemAccess.getAccessibleMenus(userId), 0L);
}

private void validateCaptcha(String captchaKey, String captcha) {
    if (!captchaEnabled) {
        return;
    }

    if (captchaKey == null || captchaKey.isBlank()) {
        throw new BusinessException("验证码Key不能为空");
    }
    if (captcha == null || captcha.isBlank()) {
        throw new BusinessException("验证码不能为空");
    }

    String key = CAPTCHA_PREFIX + captchaKey;
    String storedCode = redisTemplate.opsForValue().get(key);

    if (storedCode == null) {
        throw new BusinessException("验证码已过期");
    }
    if (!storedCode.equals(captcha.toLowerCase())) {
        throw new BusinessException("验证码错误");
    }

    redisTemplate.delete(key);
}

private LoginVO.UserInfo buildUserInfo(SystemUserAccount user) {
    return LoginVO.UserInfo.builder()
        .id(user.id())
        .username(user.username())
        .nickname(user.nickname())
        .avatar(user.avatar())
        .email(user.email())
        .phone(user.phone())
        .build();
}

private HttpServletRequest getCurrentRequest() {
    RequestAttributes attributes = RequestContextHolder.getRequestAttributes();
    if (attributes instanceof ServletRequestAttributes servletRequestAttributes) {
        return servletRequestAttributes.getRequest();
    }
    return null;
}

private String resolveUserAgent(HttpServletRequest request) {

```

```

    if (request == null) {
        return "";
    }
    String userAgent = request.getHeader(HttpHeaders.USER_AGENT);
    return userAgent == null ? "" : userAgent.trim();
}

public List<String> getUserRoles(Long userId) {
    return systemAccess.getUserRoleKeys(userId);
}

public List<String> getUserPermissions(Long userId) {
    return systemAccess.getUserPermissions(userId);
}

public List<String> getAllPermissions() {
    return systemAccess.getAllEnabledPermissions();
}

public boolean isSuperAdmin(Long userId) {
    return systemAccess.isAdmin(userId);
}

private List<MenuVO> buildMenuTree(List<SystemMenuDescriptor> allMenus, Long parentId) {
    Map<Long, List<SystemMenuDescriptor>> groupedByParent = allMenus.stream()
        .collect(Collectors.groupingBy(SystemMenuDescriptor::parentId));

    List<SystemMenuDescriptor> roots = groupedByParent.getOrDefault(parentId, new ArrayList<>());
    return roots.stream().map(menu -> toMenuVO(menu, groupedByParent)).collect(Collectors.toList());
}

private MenuVO toMenuVO(SystemMenuDescriptor menu, Map<Long, List<SystemMenuDescriptor>> groupedByParent) {
    MenuVO vo = MenuVO.builder()
        .id(menu.id())
        .menuName(menu.menuName())
        .parentId(menu.parentId())
        .menuType(menu.menuType())
        .path(menu.path())
        .component(menu.component())
        .perms(menu.perms())
        .icon(menu.icon())
        .sort(menu.sort())
        .visible(menu.visible())
        .status(menu.status())
        .build();

    List<SystemMenuDescriptor> children = groupedByParent.getOrDefault(menu.id(), new ArrayList<>());
    if (!children.isEmpty()) {
        vo.setChildren(children.stream().map(child -> toMenuVO(child, groupedByParent)).collect(Collectors.toList()));
    }
}

```

```
        return vo;
    }
}
```

=====

文件: backend/src/main/java/com/devlovecode/aiperm/modules/system/rbac/service/UserService.java

=====

```
package com.devlovecode.aiperm.modules.system.rbac.service;

import cn.dev33.satoken.stp.StpUtil;
import cn.hutool.crypto.digest.BCrypt;
import com.devlovecode.aiperm.common.domain.PageResult;
import com.devlovecode.aiperm.common.exception.BusinessException;
import com.devlovecode.aiperm.common.util.ExcelExportHelper;
import com.devlovecode.aiperm.common.util.ExcelImportHelper;
import com.devlovecode.aiperm.common.vo.ImportResultVO;
import com.devlovecode.aiperm.modules.system.rbac.dto.UserDTO;
import com.devlovecode.aiperm.modules.system.rbac.entity.SysDept;
import com.devlovecode.aiperm.modules.system.rbac.entity.SysPost;
import com.devlovecode.aiperm.modules.system.rbac.entity.SysRole;
import com.devlovecode.aiperm.modules.system.rbac.entity.SysUser;
import com.devlovecode.aiperm.modules.system.rbac.export.UserExportModel;
import com.devlovecode.aiperm.modules.system.rbac.export.UserImportModel;
import com.devlovecode.aiperm.modules.system.rbac.repository.DeptRepository;
import com.devlovecode.aiperm.modules.system.rbac.repository.PostRepository;
import com.devlovecode.aiperm.modules.system.rbac.repository.RoleRepository;
import com.devlovecode.aiperm.modules.system.rbac.repository.UserRepository;
import com.devlovecode.aiperm.modules.system.rbac.vo.UserVO;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.multipart.MultipartFile;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;
import java.util.regex.Pattern;

@Service
@RequiredArgsConstructor
```

```

public class UserService {

    private static final DateTimeFormatter DATE_TIME_FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    private static final Pattern ROLE_CODE_SEPARATOR = Pattern.compile("[, ]");

    private final UserRepository userRepo;

    private final DeptRepository deptRepo;

    private final PostRepository postRepo;

    private final RoleRepository roleRepo;

    private final ExcelExportHelper excelExportHelper;

    private final ExcellImportHelper excellImportHelper;

    /**
     * 分页查询
     */
    public PageResult<UserVO> queryPage(UserDTO dto) {
        Page<SysUser> jpaPage = userRepo.queryPage(dto.getUsername(), dto.getPhone(), dto.getDeptId(), dto.getStatus(),
            dto.getPage(), dto.getPageSize());
        return PageResult.fromJpaPage(jpaPage).map(this::toVO);
    }

    /**
     * 查询详情
     */
    public UserVO findById(Long id) {
        SysUser user = userRepo.findById(id).orElseThrow(() -> new BusinessException("用户不存在"));
        return toVO(user);
    }

    public void export(UserDTO dto, HttpServletResponse response) {
        List<UserExportModel> rows = userRepo
            .queryPage(dto.getUsername(), dto.getPhone(), dto.getDeptId(), dto.getStatus(), 1, Integer.MAX_VALUE)
            .getContent()
            .stream()
            .map(this::toVO)
            .map(this::toExportModel)
            .toList();
        excelExportHelper.export(response, "users", UserExportModel.class, rows);
    }

    public void downloadImportTemplate(HttpServletResponse response) {
        excelExportHelper.export(response, "user-import-template", UserImportModel.class, List.of());
    }
}

```

```

@Transactional
public ImportResultVO importUsers(MultipartFile file) {
    List<UserImportModel> rows = excellImportHelper.read(file, UserImportModel.class);
    ImportResultVO result = new ImportResultVO();
    int rowNumber = 2;
    for (UserImportModel row : rows) {
        try {
            importSingleUser(row);
            result.addSuccess();
        }
        catch (Exception e) {
            result.addError(rowNumber, e.getMessage());
        }
        rowNumber++;
    }
    return result;
}

/**
 * 转换为VO，填充关联数据
 */
private UserVO toVO(SysUser entity) {
    UserVO vo = new UserVO();
    vo.setId(entity.getId());
    vo.setUsername(entity.getUsername());
    vo.setNickname(entity.getNickname());
    vo.setRealName(entity.getRealName());
    vo.setEmail(entity.getEmail());
    vo.setPhone(entity.getPhone());
    vo.setGender(entity.getGender());
    vo.setAvatar(entity.getAvatar());
    vo.setDeptId(entity.getDeptId());
    vo.setStatus(entity.getStatus());
    vo.setRemark(entity.getRemark());
    vo.setCreateTime(entity.getCreateTime());
    vo.setUpdateTime(entity.getUpdateTime());

    if (entity.getDeptId() != null) {
        deptRepo.findById(entity.getDeptId()).ifPresent(dept -> vo.setDeptName(dept.getDeptName()));
    }

    if (entity.getPostId() != null) {
        postRepo.findById(entity.getPostId()).ifPresent(post -> {
            vo.setPostIds(List.of(post.getId()));
            vo.setPostNames(post.getPostName());
        });
    }

    List<Long> roleIds = userRepo.findRoleIdsByUserId(entity.getId());
    if (roleIds != null && !roleIds.isEmpty()) {

```

```

        vo.setRoleIds(roleIds);
        List<String> roleNames = new ArrayList<>();
        for (Long roleId : roleIds) {
            roleRepo.findById(roleId).ifPresent(role -> roleNames.add(role.getRoleName()));
        }
        vo.setRoleNames(String.join(" ", roleNames));
    }

    return vo;
}

/**
 * 创建
 */
@Transactional
public void create(UserDTO dto) {
    if (userRepo.existsByUsername(dto.getUsername())) {
        throw new BusinessException("用户名已存在");
    }

    SysUser entity = new SysUser();
    entity.setUsername(dto.getUsername());
    entity.setPassword(BCrypt.hashpw(dto.getPassword()));
    entity.setNickname(dto.getNickname() != null ? dto.getNickname() : dto.getUsername());
    entity.setRealName(dto.getRealName());
    entity.setEmail(dto.getEmail());
    entity.setPhone(dto.getPhone());
    entity.setGender(dto.getGender() != null ? dto.getGender() : 0);
    entity.setAvatar(dto.getAvatar());
    entity.setDeptId(dto.getDeptId());
    entity.setPostId(dto.getPostId());
    entity.setStatus(dto.getStatus() != null ? dto.getStatus() : 0);
    entity.setRemark(dto.getRemark());
    entity.setCreateBy(getCurrentUsername());
    entity.setCreateTime(LocalDateTime.now());

    userRepo.save(entity);

    if (dto.getRoleIds() != null && !dto.getRoleIds().isEmpty()) {
        userRepo.updateUserRoles(entity.getId(), dto.getRoleIds(), getCurrentUsername());
    }
}

/**
 * 更新
 */
@Transactional
public void update(Long id, UserDTO dto) {
    SysUser entity = userRepo.findById(id).orElseThrow(() -> new BusinessException("用户不存在"));

```